PPL LAB-10 EXERCISES

Date April 10, 2026

PROGRAMS USING DIFFERENT CUDA DEVICE MEMORY TYPES AND SYNCHRONIZATION

NAME : DEVADATHAN N R

ROLL NO : 04

SECTION : CSE A1

REG NO : 230905010

Sample:

Matrix multiplication of 4×4 matrix using CUDA (Tiled, Shared Memory)

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <cuda_runtime.h>

#define BLOCK_WIDTH 2
#define TILE_WIDTH 2
#define WIDTH 4

__global__ void MatMulElementThreadShared(int *a, int *b, int *c)
{
    __shared__ int MDs[TILE_WIDTH][TILE_WIDTH];
    __shared__ int NDs[TILE_WIDTH][TILE_WIDTH];

    int bx = blockIdx.x;
    int by = blockIdx.y;

    int tx = threadIdx.x;
    int ty = threadIdx.y;

    int Row = by * TILE_WIDTH + ty;
    int Col = bx * TILE_WIDTH + tx;

    int Pvalue = 0;

    for (int m = 0; m < WIDTH / TILE_WIDTH; m++)
    {
        MDs[ty][tx] = a[Row * WIDTH + m * TILE_WIDTH + tx];
        NDs[ty][tx] = b[(m * TILE_WIDTH + ty) * WIDTH + Col];

        __syncthreads();

        for (int k = 0; k < TILE_WIDTH; k++)
        {
```

```
            Pvalue += MDs[ty][k] * NDs[k][tx];
        }

        __syncthreads();
    }

    c[Row * WIDTH + Col] = Pvalue;
}

int main()
{
    int *matA, *matB, *matProd;
    int *da, *db, *dc;

    printf("\n=== Enter elements of Matrix A (4x4) ===\n");
    matA = (int *)malloc(sizeof(int) * WIDTH * WIDTH);
    for (int i = 0; i < WIDTH * WIDTH; i++)
    {
        scanf("%d", &matA[i]);
    }

    printf("\n=== Enter elements of Matrix B (4x4) ===\n");
    matB = (int *)malloc(sizeof(int) * WIDTH * WIDTH);
    for (int i = 0; i < WIDTH * WIDTH; i++)
    {
        scanf("%d", &matB[i]);
    }

    matProd = (int *)malloc(sizeof(int) * WIDTH * WIDTH);

    // Allocate device memory
    cudaMalloc((void **)&da, sizeof(int) * WIDTH * WIDTH);
    cudaMalloc((void **)&db, sizeof(int) * WIDTH * WIDTH);
    cudaMalloc((void **)&dc, sizeof(int) * WIDTH * WIDTH);

    // Copy data to device
    cudaMemcpy(da, matA, sizeof(int) * WIDTH * WIDTH, cudaMemcpyHostToDevice);
    cudaMemcpy(db, matB, sizeof(int) * WIDTH * WIDTH, cudaMemcpyHostToDevice);

    int NumBlocks = WIDTH / BLOCK_WIDTH;

    dim3 grid_conf(NumBlocks, NumBlocks);
    dim3 block_conf(BLOCK_WIDTH, BLOCK_WIDTH);

    // Kernel launch
    MatMulElementThreadShared<<<grid_conf, block_conf>>>(da, db, dc);

    // Copy result back
    cudaMemcpy(matProd, dc, sizeof(int) * WIDTH * WIDTH, cudaMemcpyDeviceToHost);

    printf("\n=== Result of Multiplication ===\n");
```

```
    for (int i = 0; i < WIDTH; i++)
    {
        for (int j = 0; j < WIDTH; j++)
        {
            printf("%d ", matProd[i * WIDTH + j]);
        }
        printf("\n");
    }

    // Free memory
    cudaFree(da);
    cudaFree(db);
    cudaFree(dc);

    free(matA);
    free(matB);
    free(matProd);

    return 0;
}
```

Output:

```
STUDENT@MIT-ICT-LAB5-4:~/Desktop/230905010/10_Lab/Mine$ nvcc l10sample.cu -o sample
STUDENT@MIT-ICT-LAB5-4:~/Desktop/230905010/10_Lab/Mine$ ./sample

=== Enter elements of Matrix A (4x4) ===
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 16

=== Enter elements of Matrix B (4x4) ===
1 0 0 0
0 1 0 0
0 0 1 0
0 0 0 1

=== Result of Multiplication ===
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 16
STUDENT@MIT-ICT-LAB5-4:~/Desktop/230905010/10_Lab/Mine$ ./sample

=== Enter elements of Matrix A (4x4) ===
1 1 1 1
1 1 1 1
1 1 1 1
1 1 1 1

=== Enter elements of Matrix B (4x4) ===
1 1 1 1
1 1 1 1
1 1 1 1
1 1 1 1

=== Result of Multiplication ===
4 4 4 4
4 4 4 4
4 4 4 4
4 4 4 4
```

Question 1:

Write a program in CUDA to perform matrix multiplication using 2D Grid and 2D Block.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <cuda_runtime.h>

// Kernel function
__global__ void matrixMulKernel(int *A, int *B, int *C, int rowsA, int colsA, int colsB)
{
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    if (row < rowsA && col < colsB)
    {
        int sum = 0;
        for (int i = 0; i < colsA; i++)
        {
            sum += A[row * colsA + i] * B[i * colsB + col];
        }
        C[row * colsB + col] = sum;
    }
}

int main()
{
    int rowsA, colsA, rowsB, colsB;

    printf("Enter rows and columns of Matrix A: ");
    scanf("%d %d", &rowsA, &colsA);

    printf("Enter rows and columns of Matrix B: ");
    scanf("%d %d", &rowsB, &colsB);

    if (colsA != rowsB)
    {
        printf("Matrix multiplication not possible!\n");
        return 0;
    }

    int *h_A = (int *)malloc(rowsA * colsA * sizeof(int));
    int *h_B = (int *)malloc(rowsB * colsB * sizeof(int));
    int *h_C = (int *)malloc(rowsA * colsB * sizeof(int));

    printf("Enter elements of Matrix A:\n");
    for (int i = 0; i < rowsA * colsA; i++)
        scanf("%d", &h_A[i]);
```

```
    printf("Enter elements of Matrix B:\n");
    for (int i = 0; i < rowsB * colsB; i++)
        scanf("%d", &h_B[i]);

    int *d_A, *d_B, *d_C;

    cudaMalloc((void **)&d_A, rowsA * colsA * sizeof(int));
    cudaMalloc((void **)&d_B, rowsB * colsB * sizeof(int));
    cudaMalloc((void **)&d_C, rowsA * colsB * sizeof(int));

    cudaMemcpy(d_A, h_A, rowsA * colsA * sizeof(int), cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B, rowsB * colsB * sizeof(int), cudaMemcpyHostToDevice);

    // 2D Block and Grid
    dim3 block(16, 16);
    dim3 grid((colsB + 15) / 16, (rowsA + 15) / 16);

    matrixMulKernel<<<grid, block>>>(d_A, d_B, d_C, rowsA, colsA, colsB);

    cudaMemcpy(h_C, d_C, rowsA * colsB * sizeof(int), cudaMemcpyDeviceToHost);

    printf("\nResult Matrix:\n");
    for (int i = 0; i < rowsA; i++)
    {
        for (int j = 0; j < colsB; j++)
            printf("%d ", h_C[i * colsB + j]);
        printf("\n");
    }

    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C);

    free(h_A);
    free(h_B);
    free(h_C);

    return 0;
}
```

Output:

```
STUDENT@MIT-ICT-LAB5-4:~/Desktop/230905010/10_Lab/Mine$ nvcc l10q1.cu -o q1
STUDENT@MIT-ICT-LAB5-4:~/Desktop/230905010/10_Lab/Mine$ ./q1
Enter rows and columns of Matrix A: 2 2
Enter rows and columns of Matrix B: 2 2
Enter elements of Matrix A:
1 2
3 4
Enter elements of Matrix B:
5 6
7 8

Result Matrix:
19 22
43 50
STUDENT@MIT-ICT-LAB5-4:~/Desktop/230905010/10_Lab/Mine$ ./q1
Enter rows and columns of Matrix A: 2 3
Enter rows and columns of Matrix B: 3 2
Enter elements of Matrix A:
1 2 3
4 5 6
Enter elements of Matrix B:

7 8
9 10
11 12

Result Matrix:
58 64
139 154
```

Question 2:

Write a program in CUDA to improve the performance of 1D parallel convolution using constant memory.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <cuda_runtime.h>

#define MAX_MASK_WIDTH 64

// Constant memory for mask
__constant__ int d_M[MAX_MASK_WIDTH];

// Kernel
__global__ void conv1D(int *N, int *P, int width, int mask_width)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int radius = mask_width / 2;

    if (i < width)
    {
        int sum = 0;

        for (int j = 0; j < mask_width; j++)
        {
```

```
            int idx = i - radius + j;

            if (idx >= 0 && idx < width)
            {
                sum += N[idx] * d_M[j];
            }
        }

        P[i] = sum;
    }
}

int main()
{
    int width, mask_width;

    printf("Enter size of input array: ");
    scanf("%d", &width);

    printf("Enter mask width (<= %d): ", MAX_MASK_WIDTH);
    scanf("%d", &mask_width);

    int *h_N = (int *)malloc(width * sizeof(int));
    int *h_P = (int *)malloc(width * sizeof(int));
    int *h_M = (int *)malloc(mask_width * sizeof(int));

    printf("Enter input array:\n");
    for (int i = 0; i < width; i++)
        scanf("%d", &h_N[i]);

    printf("Enter mask array:\n");
    for (int i = 0; i < mask_width; i++)
        scanf("%d", &h_M[i]);

    int *d_N, *d_P;

    cudaMalloc((void **)&d_N, width * sizeof(int));
    cudaMalloc((void **)&d_P, width * sizeof(int));

    cudaMemcpy(d_N, h_N, width * sizeof(int), cudaMemcpyHostToDevice);

    // Copy mask to constant memory
    cudaMemcpyToSymbol(d_M, h_M, mask_width * sizeof(int));

    int blockSize = 256;
    int gridSize = (width + blockSize - 1) / blockSize;

    conv1D<<<gridSize, blockSize>>>(d_N, d_P, width, mask_width);

    cudaMemcpy(h_P, d_P, width * sizeof(int), cudaMemcpyDeviceToHost);
```

```
    printf("\nOutput array:\n");
    for (int i = 0; i < width; i++)
        printf("%d ", h_P[i]);

    printf("\n");

    cudaFree(d_N);
    cudaFree(d_P);

    free(h_N);
    free(h_P);
    free(h_M);

    return 0;
}
```

Output:

```
STUDENT@MIT-ICT-LAB5-4:~/Desktop/230905010/10_Lab/Mine$ nvcc l10q2.cu -o q2
STUDENT@MIT-ICT-LAB5-4:~/Desktop/230905010/10_Lab/Mine$ ./q2
Enter size of input array: 5
Enter mask width (<= 64):
3
Enter input array:
1 2 3 4 5
Enter mask array:
1 1 1

Output array:
3 6 9 12 9
STUDENT@MIT-ICT-LAB5-4:~/Desktop/230905010/10_Lab/Mine$ ./q2
Enter size of input array: 5
Enter mask width (<= 64): 3
Enter input array:
1 2 3 4 5
Enter mask array:
1 0 -1

Output array:
-2 -2 -2 -2 4
STUDENT@MIT-ICT-LAB5-4:~/Desktop/230905010/10_Lab/Mine$ ./q2
Enter size of input array: 4
Enter mask width (<= 64): 1
Enter input array:
5 6 7 8
Enter mask array:
2

Output array:
10 12 14 16
```

Question 3:

Write a program in CUDA to perform inclusive scan algorithm.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <cuda_runtime.h>

// Kernel for Inclusive Scan (Hillis-Steele)
__global__ void inclusiveScanKernel(int *input, int *output, int n)
{
    extern __shared__ int temp[];

    int tid = threadIdx.x;

    // Load input into shared memory
    if (tid < n)
        temp[tid] = input[tid];
    __syncthreads();

    // Inclusive Scan
    for (int offset = 1; offset < n; offset *= 2)
    {
        int val = 0;

        if (tid >= offset)
            val = temp[tid - offset];

        __syncthreads();

        if (tid < n)
            temp[tid] += val;

        __syncthreads();
    }

    // Write result
    if (tid < n)
        output[tid] = temp[tid];
}

int main()
{
    int n;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int *h_input = (int *)malloc(n * sizeof(int));
    int *h_output = (int *)malloc(n * sizeof(int));

    printf("Enter elements:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &h_input[i]);
```

```
    int *d_input, *d_output;

    cudaMalloc((void **)&d_input, n * sizeof(int));
    cudaMalloc((void **)&d_output, n * sizeof(int));

    cudaMemcpy(d_input, h_input, n * sizeof(int), cudaMemcpyHostToDevice);

    // Launch kernel (single block version)
    inclusiveScanKernel<<<1, n, n * sizeof(int)>>>(d_input, d_output, n);

    cudaMemcpy(h_output, d_output, n * sizeof(int), cudaMemcpyDeviceToHost);

    printf("\nInclusive Scan Output:\n");
    for (int i = 0; i < n; i++)
        printf("%d ", h_output[i]);

    printf("\n");

    cudaFree(d_input);
    cudaFree(d_output);

    free(h_input);
    free(h_output);

    return 0;
}
```

Output: